
Rapport Technique

Service de Localisation GPS

Microservice Node.js/TypeScript — gRPC, TimescaleDB, Kafka et
Géofencing

Système de Gestion de Flotte de Véhicules

Master 1 GIL — 2025/2026

Informations du projet

Auteurs :	Kherbachi Rima & Mouhoubi Leiticia
Formation :	Master 1 GIL
Université :	Université de Rouen Normandie
Encadrants :	Lydia et Luc
Année :	2025 – 2026
Semaine documentée :	5

Mars – Avril 2026

Table des matières

1	Introduction et Architecture	2
1.1	Présentation du livrable	2
1.2	Structure du projet	2
1.3	Diagramme de flux	3
2	Authentification SSO — Keycloak	4
2.1	Configuration Keycloak	4
2.2	Intercepteur JWT Axios	5
3	Contrôle d'Accès Basé sur les Rôles (RBAC)	7
3.1	Rôles et permissions	7
3.2	Composant ProtectedRoute	7
3.3	Routage RBAC dans App.tsx	8
4	Pages et Fonctionnalités	9
4.1	Tableau de bord (Dashboard)	9
4.2	Gestion des Véhicules	9
4.3	Gestion des Conducteurs	10
4.4	Carte de Localisation (Leaflet)	10
4.5	Gestion Maintenance	12
5	Composants Partagés	14
5.1	StatusBadge	14
5.2	Modal	14
5.3	Design System — Thème Sombre	15
6	Tests End-to-End avec Cypress	16
6.1	Stratégie de tests	16
6.2	Commandes personnalisées	16
6.3	Exemples de cas de tests	17
6.4	Commandes d'exécution	18
7	Déploiement et Points Techniques	19
7.1	Variables d'environnement	19
7.2	Difficultés rencontrées et solutions	19
7.3	Construction et démarrage	19
	Conclusion	21

Chapitre 1

Introduction et Architecture

1.1 Présentation du livrable

La semaine 6 est consacrée au développement du **microfrontend React**, l'interface graphique unifiée du système de gestion de flotte. Elle consolide l'ensemble des microservices développés lors des semaines précédentes en les exposant à travers une **Single Page Application** (SPA) sécurisée, accessible selon les rôles métier.

Technologie	Version	Rôle
React	19.2	Framework UI (composants fonctionnels + hooks)
TypeScript	~6.0	Typage statique
Vite	8.0	Bundler / dev server (HMR)
keycloak-js	26.2	Client SSO OIDC / PKCE S256
react-leaflet	5.0	Carte interactive GPS (React 19 compatible)
Leaflet	1.9	Moteur cartographique OpenStreetMap
Recharts	3.8	Graphiques SVG (vitesse, activité)
socket.io-client	4.8	WebSocket temps réel (positions GPS)
Axios	1.15	Client HTTP avec intercepteur JWT
lucide-react	1.8	Icônes SVG
Cypress	15.13	Tests end-to-end

TABLE 1.1 – Pile technologique du microfrontend

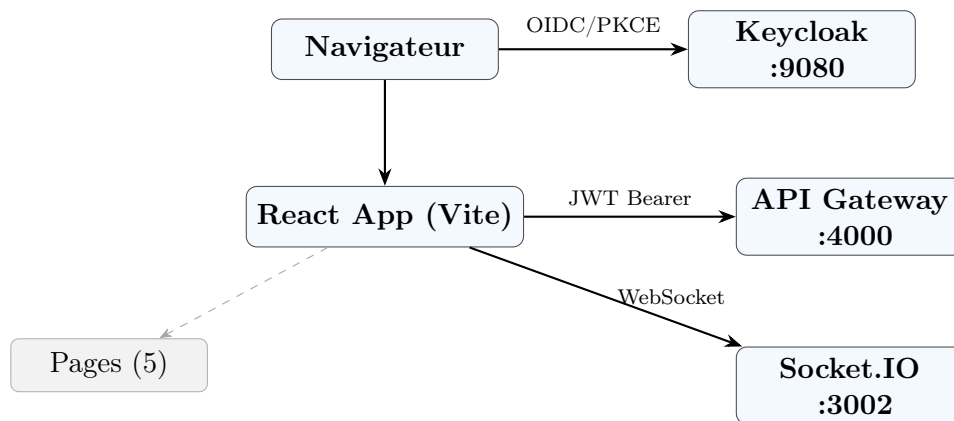
1.2 Structure du projet

```
1 src/
2     auth.ts      # Initialisation Keycloak (PKCE S256)
3     api.ts       # Axios + intercepteur JWT auto-refresh
4     App.tsx      # Router + Sidebar + RBAC routing
5     App.css      # Theme sombre + variables CSS
6     index.css    # Reset global + scrollbar personnalisée
7     main.tsx     # Point d'entrée (initKeycloak + CSS Leaflet)
8     components/
9         ProtectedRoute.tsx # Garde RBAC par rôle Keycloak
10        StatusBadge.tsx   # Badge color par statut
11
12     m t i e r
```

```
11      Modal.tsx      # Fen tre modale g n rique (ESC +  
    overlay)  
12      pages/  
13          Dashboard.tsx      # KPI Socket.IO + Recharts  
14          Vehicules.tsx      # CRUD v hicules + filtres  
15          Conducteurs.tsx      # CRUD conducteurs (grille de  
    cartes)  
16          Localisation.tsx      # Carte Leaflet temps r el  
17          Maintenance.tsx      # Interventions + workflow  
    statuts  
18          Unauthorized.tsx      # Page 403 (Acc s refus RBAC)
```

Listing 1.1 – Arborescence du microfrontend (src/)

1.3 Diagramme de flux



Chapitre 2

Authentication SSO — Keycloak

2.1 Configuration Keycloak

L'authentification repose sur **Keycloak 26** avec le flux Authorization Code + **PKCE S256**, garantissant qu'aucun secret client n'est exposé dans le code frontend.

Paramètre	Valeur
Realm	fleet-management
Client ID	fleet-api-gateway
URL	http://localhost:9080
Flow	Authorization Code + PKCE S256
onLoad	login-required

TABLE 2.1 – Paramètres de configuration Keycloak

```
1 import Keycloak from 'keycloak-js';
2
3 const keycloak = new Keycloak({
4   url:      'http://localhost:9080',
5   realm:    'fleet-management',
6   clientId: 'fleet-api-gateway',
7 });
8
9 export const initKeycloak = (onAuthenticatedCallback: () => void)
10 => {
11   keycloak
12     .init({ onLoad: 'login-required',
13               checkLoginIframe: false,
14               pkceMethod: 'S256' })
15     .then(authenticated => {
16       if (!authenticated) console.warn('Non authentifié');
17       onAuthenticatedCallback();
18     });
19 };
```

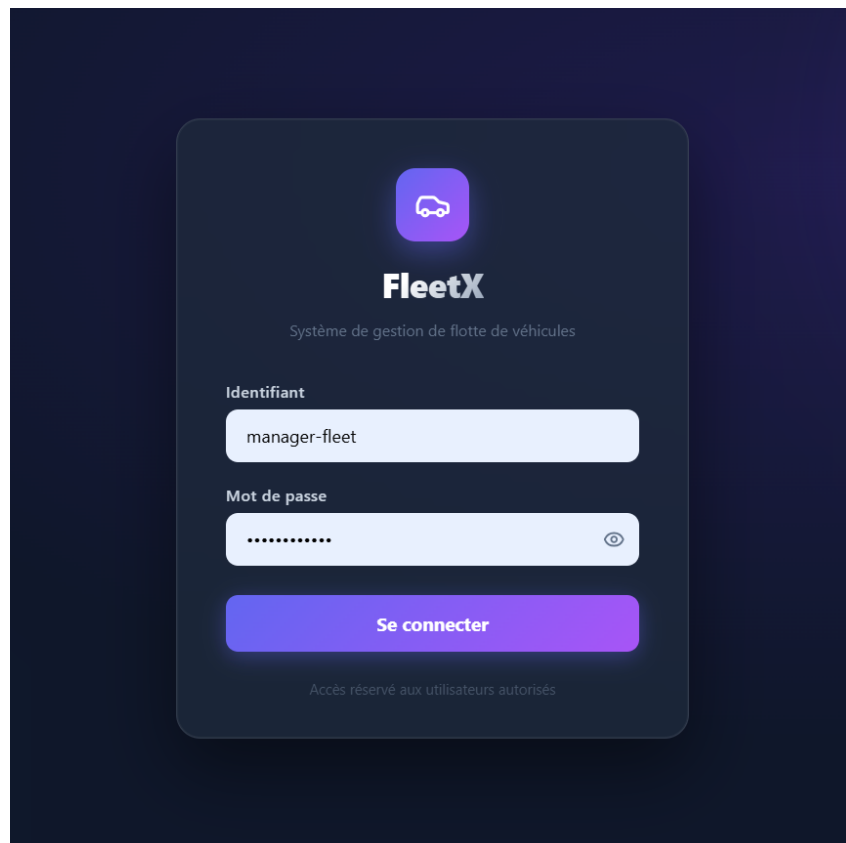


FIGURE 2.1 – Page de connexion avec Keycloak.

```
20 export default keycloak;
```

Listing 2.1 – src/auth.ts — Initialisation Keycloak

2.2 Intercepteur JWT Axios

Chaque requête HTTP est automatiquement enrichie avec le token Bearer. En cas de réponse 401, le token est renouvelé silencieusement via `keycloak.updateToken()`, sans interruption pour l'utilisateur.

```
1  const api = axios.create({ baseURL: 'http://localhost:4000' });
2
3  api.interceptors.request.use(async (config) => {
4    await keycloak.updateToken(70); // Renouvelle si expiration < 70s
5    config.headers.Authorization = `Bearer ${keycloak.token}`;
6    return config;
7  });
8
9  api.interceptors.response.use(
10   res => res,
11   async (error) => {
12     if (error.response?.status === 401) {
13       await keycloak.updateToken(-1); // Force le renouvellement
```

```
14     }  
15     return Promise.reject(error);  
16   }  
17 );
```

Listing 2.2 – `src/api.ts` — Intercepteur JWT avec auto-refresh

Chapitre 3

Contrôle d'Accès Basé sur les Rôles (RBAC)

3.1 Rôles et permissions

Quatre rôles Keycloak de niveau *realm* sont définis. Chaque rôle détermine les pages accessibles et les actions autorisées.

TABLE 3.1 – Matrice des rôles et permissions

Rôle	Permissions	Pages accessibles
admin	Accès complet, tous CRUD	Toutes
manager	Gestion véhicules + conducteurs	Dashboard, Véhicules, Conducteurs, Localisation, Maintenance
technicien	Lecture véhicules, gestion maintenance	Dashboard, Véhicules, Localisation, Maintenance
utilisateur	Lecture seule	Dashboard, Véhicules, Localisation

TABLE 3.2 – Matrice des rôles et permissions

3.2 Composant ProtectedRoute

Le composant `ProtectedRoute` encapsule la logique de garde. Il interroge `keycloak.hasRealmRole()` pour chaque rôle autorisé, et redirige vers `/unauthorized` (page 403) si aucun ne correspond.

```
1 import { Navigate } from 'react-router-dom';
2 import keycloak from '../auth';
3
4 interface ProtectedRouteProps {
5   children: React.ReactNode;
6   roles: string[];
7 }
8
```

```
9  const ProtectedRoute = ({ children, roles }: ProtectedRouteProps)
    => {
10   const hasAccess = roles.some(role => keycloak.hasRealmRole(role))
    ;
11   if (!hasAccess) return <Navigate to="/unauthorized" replace />;
12   return <>{children}</>;
13 };
14
15 export default ProtectedRoute;
```

Listing 3.1 – src/components/ProtectedRoute.tsx

3.3 Routage RBAC dans App.tsx

```
1  <Routes>
2    <Route path="/" element={<Dashboard />} />
3    <Route path="/unauthorized" element={<Unauthorized />} />
4
5    <Route path="/vehicules" element={
6      <ProtectedRoute roles={['admin', 'manager', 'technicien', '
7        utilisateur']}>
8        <Vehicules />
9      </ProtectedRoute>
10   } />
11   <Route path="/conducteurs" element={
12     <ProtectedRoute roles={['admin', 'manager']}>
13       <Conducteurs />
14     </ProtectedRoute>
15   } />
16   <Route path="/localisation" element={
17     <ProtectedRoute roles={['admin', 'manager', 'technicien', '
18       utilisateur']}>
19       <Localisation />
20     </ProtectedRoute>
21   } />
22   <Route path="/maintenance" element={
23     <ProtectedRoute roles={['admin', 'manager', 'technicien']}>
24       <Maintenance />
25     </ProtectedRoute>
26   } />
27 </Routes>
```

Listing 3.2 – Extrait des routes protegees dans App.tsx

Chapitre 4

Pages et Fonctionnalités

4.1 Tableau de bord (Dashboard)

Le Dashboard agrège les métriques en temps réel reçues via **Socket.IO** depuis le `localisation-service` (port 3002).

Indicateur	Source
Nombre de véhicules actifs	Socket.IO — positions des 5 dernières minutes
Vitesse moyenne (km/h)	Calcul sur les 20 derniers événements
Total événements	Compteur cumulé des positions reçues
Graphique de vitesse	Recharts AreaChart SVG
Flux d'activité	Liste des 10 dernières positions horodatées

TABLE 4.1 – Indicateurs du tableau de bord

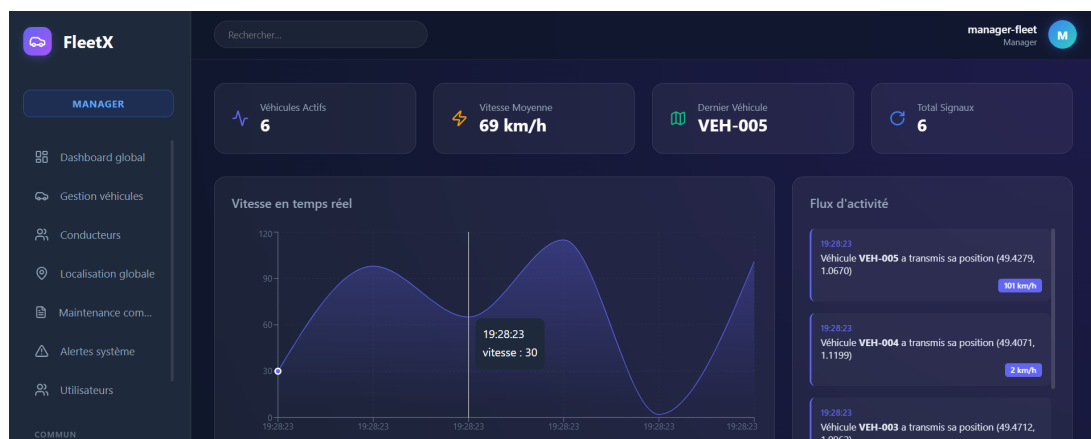


FIGURE 4.1 – Tableau de bord — KPI temps réel et graphique de vitesse

4.2 Gestion des Véhicules

Interface CRUD complète exposant le `vehicule-service` via l'API Gateway.

- **Tableau** avec colonnes : Immatriculation, Marque/Modèle, Année, Statut, Actif, Actions

- **Filtres** par statut : Tous / DISPONIBLE / EN_COURSE / EN_MAINTENANCE
- **Changement de statut** inline via <select> (admin/manager uniquement)
- **Modal création** : immatriculation, marque, modèle, année
- **Modal édition** : modification de toutes les propriétés
- **Suppression logique** : DELETE /vehicules/{id} — désactivation du champ actif

Méthode	Endpoint	Action
GET	/vehicules	Liste des véhicules
POST	/vehicules	Créer un véhicule
PUT	/vehicules/{id}	Mettre à jour
PATCH	/vehicules/{id}/statut	Changer le statut
DELETE	/vehicules/{id}	Désactivation logique

TABLE 4.2 – Endpoints REST — vehicule-service

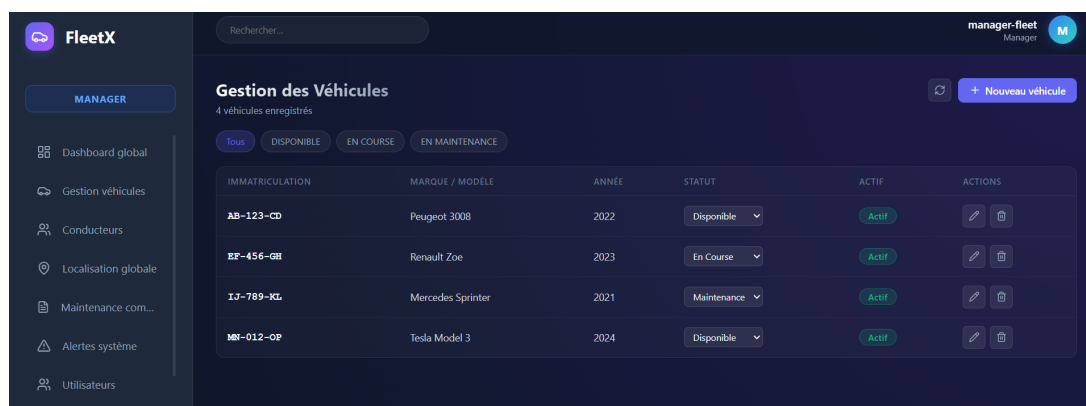


FIGURE 4.2 – Page Véhicules — liste avec badges de statut

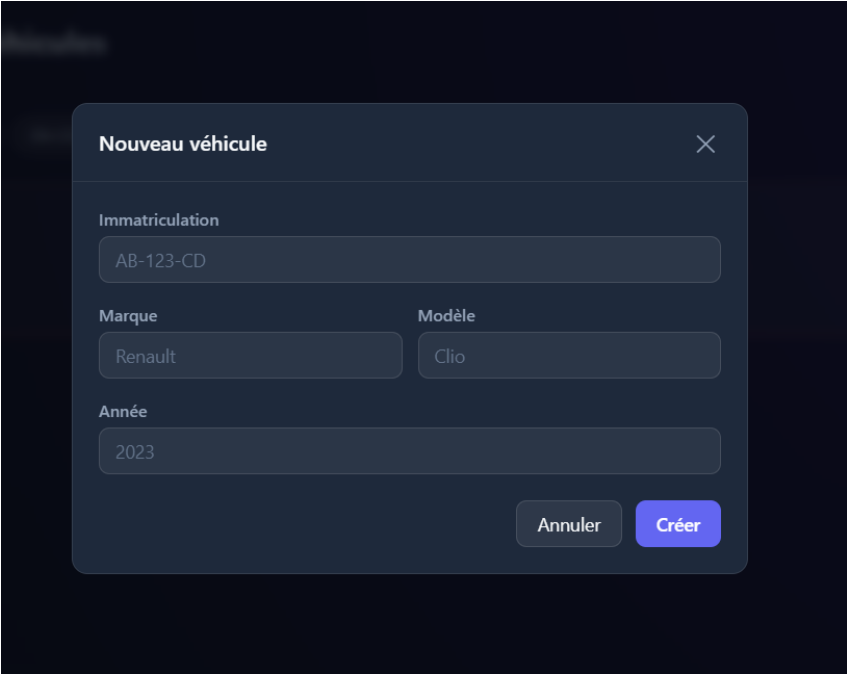
4.3 Gestion des Conducteurs

Interface en **grille de cartes** avec avatar généré à partir des initiales. Chaque carte affiche les informations du permis avec un code couleur sur la date d'expiration :

- **Vert** si l'expiration est à plus de 30 jours
- **Orange** si l'expiration est dans moins de 30 jours
- **Rouge** si le permis est expiré

Une barre de recherche filtre les conducteurs en temps réel (côté client) par nom ou email.

4.4 Carte de Localisation (Leaflet)



Modal de création d'un véhicule. Le formulaire contient les champs suivants :

- Immatriculation: AB-123-CD
- Marque: Renault
- Modèle: Clio
- Année: 2023

Les boutons "Annuler" et "Créer" sont situés en bas à droite.

FIGURE 4.3 – Modal de création d'un véhicule

```

1 // Les bundlers ES cassent le mecanisme interne de Leaflet
2 delete (L.Icon.Default.prototype as unknown as Record<string,
   unknown>)
3   ._getIconUrl;
4 L.Icon.Default.mergeOptions({
5   iconUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon
   .png',
6   iconRetinaUrl:
7     'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon-2x.png
   ',
8   shadowUrl:
9     'https://unpkg.com/leaflet@1.9.4/dist/images/marker-shadow.png'
10  });

```

Listing 4.1 – Fix icone Leaflet avec Vite (bundler ES modules)

Fonctionnalité	Implémentation
Fond de carte	OpenStreetMap (tuiles libres, sans clé API)
Centre initial	Rouen (lat 49.4431, lng 1.0993)
Marqueurs colorés	DivIcon HTML — vert/bleu/orange par statut
Popup au clic	Immatriculation, statut, vitesse, heure
Zones géofencing	Composant <Circle> avec remplissage semi-transparent
Ajustement automatique	Hook useMap() + fitBounds sur positions
Temps réel	Socket.IO — mise à jour des marqueurs à chaque position
Indicateur connexion	Badge connecté/hors ligne (état WebSocket)

TABLE 4.3 – Fonctionnalités de la carte Leaflet

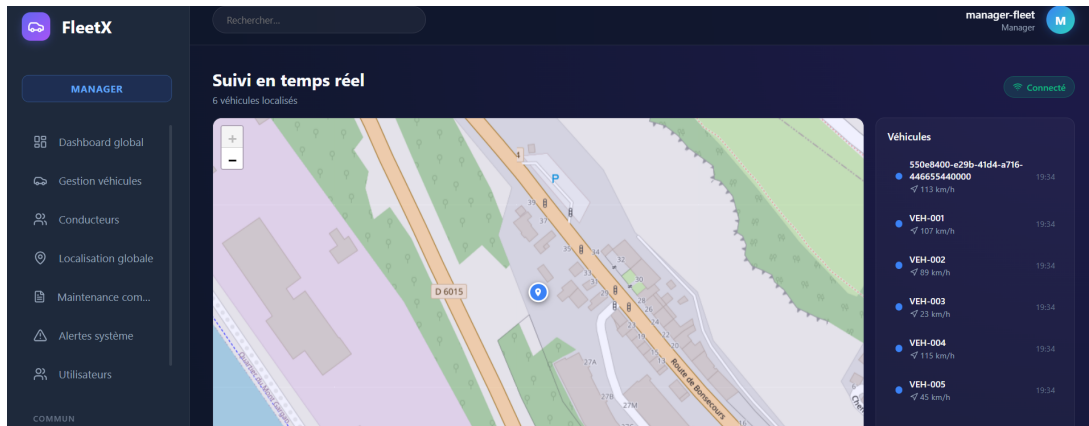


FIGURE 4.4 – Carte Leaflet — marqueurs GPS colorés et zones de géofencing

```

1 function FitBounds({ positions }: { positions: VehiculePosition[]
2   }) {
3   const map = useMap();
4   useEffect(() => {
5     if (positions.length === 0) return;
6     const bounds = L.latLngBounds(
7       positions.map(p => [p.latitude, p.longitude])
8     );
9     map.fitBounds(bounds, { padding: [40, 40] });
10  }, []);
11  return null;
12 }

```

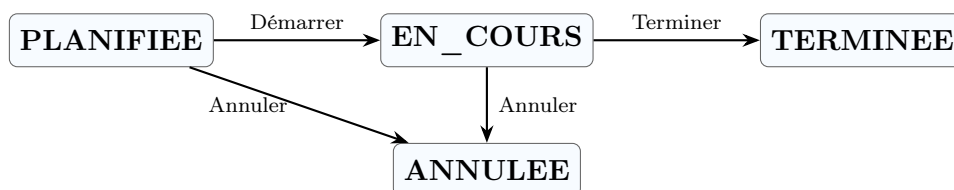
Listing 4.2 – Composant FitBounds — ajustement automatique du zoom

4.5 Gestion Maintenance

Méthode	Endpoint	Action
GET	/maintenance/interventions	Liste des interventions
POST	/maintenance/interventions	Créer une intervention
PATCH	/maintenance/interventions/{id}/statut	Changer le statut

TABLE 4.4 – Endpoints REST — maintenance-service

Le workflow des statuts est piloté par des boutons contextuels :



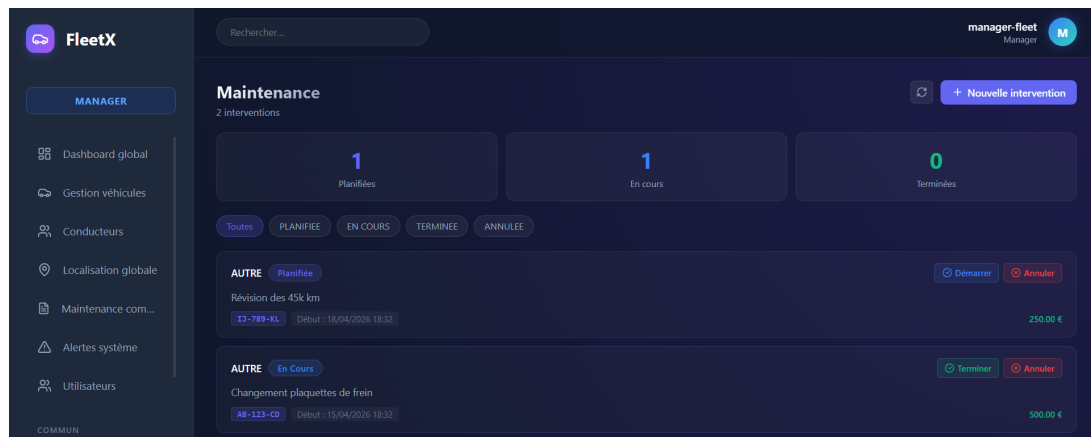


FIGURE 4.5 – Page Maintenance — KPI et liste des interventions

Chapitre 5

Composants Partagés

5.1 StatusBadge

Composant générique qui affiche un badge coloré pour tout statut métier. Il accepte une chaîne ou un booléen et dispose d'un fallback gris pour les valeurs inconnues.

Clé	Label affiché	Couleur
DISPONIBLE	Disponible	Vert #10b981
EN_COURSE	En Course	Bleu #3b82f6
EN_MAINTENANCE	Maintenance	Orange #f59e0b
PLANIFIEE	Planifiée	Violet #6366f1
EN_COURS	En Cours	Bleu #3b82f6
TERMINEE	Terminée	Vert #10b981
ANNULEE	Annulée	Rouge #ef4444
PENDING	En attente	Orange #f59e0b
VALIDE	Validée	Vert #10b981
REJETEE	Rejetée	Rouge #ef4444
true	Actif	Vert #10b981
false	Inactif	Rouge #ef4444

TABLE 5.1 – Configuration du composant StatusBadge

5.2 Modal

Dialog générique accessible :

- Fermeture par touche **Échap** (listener sur `document`)
- Fermeture par **clic sur l'overlay** semi-transparent
- Fermeture par bouton dans l'en-tête
- Largeur configurable via la prop `width` (défaut : 520px)
- Attributs `data-testid` sur tous les éléments pour Cypress
- Fond avec `backdrop-filter: blur(4px)`

5.3 Design System — Thème Sombre

```
1 :root {  
2   --primary:      #6366f1;           /* Indigo */  
3   --bg-dark:      #0f172a;          /* Fond app */  
4   --bg-sidebar:   #1e293b;          /* Fond sidebar */  
5   --glass:        rgba(255,255,255,0.05);  
6   --border-glass: rgba(255,255,255,0.1);  
7   --text-main:    #f8fafc;  
8   --text-muted:   #94a3b8;  
9 }
```

Listing 5.1 – Variables CSS du design system (App.css)

Le layout principal utilise `display: grid` avec deux colonnes : 240px pour la sidebar et 1fr pour le contenu.

Chapitre 6

Tests End-to-End avec Cypress

6.1 Stratégie de tests

Fichier	Tests	Scénarios couverts
01_auth.cy.ts	9	Sidebar, topbar, navigation, liens
02_dashboard.cy.ts	4	KPI, Recharts SVG, WebSocket
03_vehicules.cy.ts	10	CRUD, filtres, modal (ouverture/fermeture/soumission)
04_conducteurs.cy.ts	9	Grille, recherche, CRUD, cartes
05_localisation.cy.ts	7	Carte Leaflet, WebSocket, sélection véhicule
06_maintenance.cy.ts	13	KPI, filtres, workflow statuts, CRUD
07_rbac.cy.ts	7	Page 403, navigation active, routes inconnues
08_status_badge.cy.ts	8	Labels, couleurs, badges par statut
Total	67	

TABLE 6.1 – Couverture de la suite Cypress — 67 tests end-to-end

6.2 Commandes personnalisées

Deux commandes sont définies dans `cypress/support/commands.ts` :

```
1 // cy.login(role)      stub Keycloak sans serveur SSO
2 Cypress.Commands.add('login',
3   (role: 'admin' | 'manager' | 'technicien' | 'utilisateur' = '
4     admin') => {
5     cy.window().then((win) => {
6       (win as any).__KC_STUB__ = {
7         authenticated: true,
8         token: 'fake-jwt-token',
9         tokenParsed: { preferred_username: 'test-${role}' },
10        hasRealmRole: (r: string) => r === role,
11        logout: () => {},
12        updateToken: () => Promise.resolve(true),
13      };
14    });
15  });
```

```

16
17 //cy.mockApi()      intercept REST avec fixtures JSON
18 Cypress.Commands.add('mockApi', () => {
19   cy.intercept('GET', '**/vehicules',
20     { fixture: 'vehicules.json' }).as('getVehicules');
21   cy.intercept('GET', '**/conducteurs',
22     { fixture: 'conducteurs.json' }).as('getConducteurs');
23   cy.intercept('GET', '**/maintenance/interventions',
24     { fixture: 'maintenance.json' }).as('getInterventions');
25 });

```

Listing 6.1 – Commandes personnalisées Cypress

6.3 Exemples de cas de tests

```

1 it('filtre par statut DISPONIBLE', () => {
2   cy.contains('button', 'DISPONIBLE').click();
3   cy.contains('AB-123-CD').should('be.visible');
4   cy.contains('IJ-789-KL').should('not.exist');
5 });
6
7 it('ouvre le modal de creation via le bouton Nouveau', () => {
8   cy.get('[data-testid="btn-create-vehicule"]').click();
9   cy.get('[data-testid="modal-content"]').should('be.visible');
10  cy.contains('Nouveau vehicule').should('be.visible');
11 });
12
13 it('ferme le modal avec la touche Echap', () => {
14   cy.get('[data-testid="btn-create-vehicule"]').click();
15   cy.get('body').type('{esc}');
16   cy.get('[data-testid="modal-content"]').should('not.exist');
17 });
18
19 it('soumet le formulaire de creation', () => {
20   cy.intercept('POST', '**/vehicules',
21     { statusCode: 201, body: { id: 'veh-new' } }).as('
22     createVehicule');
23
24   cy.get('[data-testid="btn-create-vehicule"]').click();
25   cy.get('[data-testid="input-immatriculation"]').type('QR-999-ST')
26   ;
27   cy.get('[data-testid="input-marque"]').type('Toyota');
28   cy.get('[data-testid="input-modele"]').type('Yaris');
29   cy.get('[data-testid="input-annee"]').type('2024');
30   cy.get('[data-testid="create-vehicule-form"]').submit();
31
32   cy.wait('@createVehicule').its('request.body')
33     .should('deep.include', { immatriculation: 'QR-999-ST' });
34 });

```

Listing 6.2 – Tests CRUD Vehicules (extrait)

6.4 Commandes d'exécution

```
1 # Mode interactif (interface graphique)
2 npx cypress open
3
4 # Mode headless (CI/CD)
5 npx cypress run
6
7 # Fichier spécifique
8 npx cypress run --spec "cypress/e2e/03_vehicules.cy.ts"
9
10 # Avec rapport JUnit (integration CI)
11 npx cypress run --reporter junit
```

Listing 6.3 – Lancement des tests Cypress

Chapitre 7

Déploiement et Points Techniques

7.1 Variables d'environnement

TABLE 7.1 – Variables d'environnement Vite du frontend

Variable	Valeur par défaut	Requis
VITE_API_URL	http://localhost:4000	Non
VITE_WS_URL	http://localhost:3002	Non
VITE_KC_URL	http://localhost:9080	Oui
VITE_KC_REALM	fleet-management	Oui
VITE_KC_CLIENT	fleet-api-gateway	Oui

7.2 Difficultés rencontrées et solutions

TABLE 7.2 – Difficultés rencontrées et solutions apportées

Difficulté	Solution apportée
react-leaflet@4.x incompatible React 19	Migration vers react-leaflet@5.0 (React 19 natif)
index.css Vite fixait #root à 1126px	Remplacement par reset minimal : width: 100%; height: 100vh
Carte Leaflet non stylisée (CSS absent)	import 'leaflet/dist/leaflet.css' dans main.tsx avant l'app CSS
Icônes Leaflet manquantes avec Vite (bundler ES)	Suppression de _getIconUrl + mergeOptions avec URLs CDN
Cypress ne peut pas contourner Keycloak	Commande cy.login(role) injectant un stub sur window
TypeScript 6 strict : cast Leaflet	as unknown as Record<string, unknown> pour le prototype

7.3 Construction et démarrage

```
1 # Installation des dependances
2 npm install
3
4 # Serveur de developpement (http://localhost:5173)
5 npm run dev
6
7 # Build de production (TypeScript + Vite)
8 npm run build
9
10 # Previsualisation du build
11 npm run preview
12
13 # Lint ESLint
14 npm run lint
```

Listing 7.1 – Scripts npm disponibles

Conclusion

La semaine 6 marque la livraison du **microfrontend React**, interface unifiée du système FleetX. Elle consolide cinq semaines de développement microservices en les exposant à travers une SPA professionnelle, sécurisée et testée.

Réalisations principales :

- SPA React 19 complète avec thème sombre et design system cohérent.
- SSO Keycloak 26 (PKCE S256, *login-required*) — zéro secret côté client.
- RBAC : 4 rôles, routes protégées, page 403.
- Dashboard temps réel Socket.IO + graphiques Recharts.
- Carte Leaflet avec marqueurs GPS colorés, géofencing et ajustement automatique.
- CRUD Véhicules (tableau + filtres + modals + workflow statuts).
- CRUD Conducteurs (grille de cartes + recherche + indicateur permis).
- Gestion Maintenance (workflow PLANIFIEE → EN_COURS → TERMINEE/ANNULEE).
- **67 tests Cypress end-to-end** répartis en 8 fichiers.
- Composants partagés réutilisables : StatusBadge, Modal, ProtectedRoute.

Avancement global du projet : 6/7 semaines